

Three repairs for greedy layer-wise Tsetlin training

Testing the proposals of §7.3, on CIFAR-10 and on tasks whose ceilings are known

Vajira Thambawita

Simula Metropolitan Center for Digital Engineering

Experiments run with `torchTsetlin` 0.1.3

Summary

A previous report found the multilayer convolutional Tsetlin machine's *architecture* sound and its greedy supervised layer-wise *training* not, to the point where random first-layer clauses beat trained ones on CIFAR-10, and diagnosed the cause as density: a layer trained to classify fires on 0.14% of patch positions, and a second layer over bits that sparse can only learn negations. Its §7.3 named three repairs. This report implements and measures all three, at the same architecture, budget and seeds, with the acceptance criteria fixed before any run.

The diagnosis was right and the repair is decisive. A two-sided controller that walks each clause towards a target firing rate — no labels, no training, seconds of work — lifts the stack from 15.37% to 24.46% over the trained first layer, and to 35.59% once it is also told to cut each clause to 6 of its 85 literals. Run over an *untrained* first layer it reaches 42.62%, against 35.44% and 37.76% for the two single-layer baselines at the same clause budget — the first configuration in either report for which the second layer is worth what it costs.

An unsupervised objective wins where the architecture matters. A patch autoencoder (A) reaches 99.9% on a relational task where one layer is at chance by construction and 87.9% where one layer is capped at 75%, both past the first report's *hand-built oracle*, whose exact detectors turn out to be too sparse to conjoin. Over a repaired first layer a layer-2 clause shrinks from 1600 literals to 5 — the composition the proposal predicted, and an interpretable one. **Credit propagation (c) does not survive implementation:** no forgetting term, then a forgetting term that erases the layer, then no bandwidth, and finally drift. What both reports now say is that the architecture and the density diagnosis were right, and the part to discard is supervised layer-wise training itself.

A follow-up to *Multilayer Convolutional Tsetlin Machines: an evaluation of greedy layer-wise clause stacking on CIFAR-10*, whose §7.3 named the three ideas tested here. All figures and every number in this report are generated from `experiments/mctm/results/*.json` by `report_data_ideas.py`. Nothing is hand-typed.

Contents

1	The question this report answers	1
1.1	What this report adds	1
2	Implementation	2
2.1	A — a patch autoencoder for layer 1	2
2.2	B — a two-sided density controller	2
2.3	C — credit propagation	3
3	Experimental setup	4
3.1	What would count as success, fixed before the runs	4
3.2	The arms	4
4	E3: a different objective for layer 1	5
4.1	Result: the statistics change exactly as predicted	6
4.2	And on CIFAR-10 the stack still falls short	6
4.3	What the readout says, and what it cannot say	6
5	E4: explicit density calibration	7
5.1	Density is a real effect, and it is not the whole effect	8
5.2	Testing the residual: a second target, on clause size	9
5.3	The same controller on an untrained layer 1	10
5.4	And when combined with A	11
6	E5: credit propagation	11
6.1	Rung 1: the rule as written has no forgetting term	12
6.2	Rung 2: Type 1b has to be blamed, not sampled — and it erases layer 1	12
6.3	Rung 3: balancing the two directions, and what that exposes	12
6.4	Rung 4: warm start, and the bandwidth result at its sharpest	13
6.5	What the ladder actually shows	13
7	E6: the tasks whose ceilings are known	14
7.1	near: the repairs work, completely	14
7.2	xor: the same winner, and a limit on the controller	16
7.3	What the second layer becomes	16
7.4	Credit propagation fails here too, and in the same way	17
8	Discussion	17
8.1	Verdict against the criteria fixed in advance	17
8.2	What made the difference	17
8.3	What the second layer is worth, and where	18
8.4	What a feature layer has to satisfy	18
8.5	What to do next	18
8.6	Limitations	19
9	Reproducing this report	20

1 The question this report answers

The previous report evaluated the multilayer convolutional Tsetlin machine — convolutional layers that keep their spatial clause-activation maps instead of OR-pooling them away, trained greedily and frozen — and split cleanly into a positive and a negative result.

In short. The architecture was vindicated. On a task whose one-layer ceiling is known analytically, a two-layer stack given a hand-built first layer cleared it (78.5% against 75% at a matched channel count); on a relational task the variant that keeps the spatial map reached 76.9% while the global-pooling stack was pinned at chance whatever features it was given.

The training scheme was not. On CIFAR-10 the stack reached 15.37% against 35.44% for a single layer of the same total size, and fell below the 34.36% its own frozen first layer achieved alone. *Random* first-layer clauses beat trained ones, 37.60% against 15.37%.

The diagnosis was a coupling. Training layer 1 to classify rewards *specific* clauses — many included literals, rare firing — while layer 2 needs *general* ones whose activations are common enough to be conjoined. Layer 1 fires on a fraction of a percent of patch positions, and layer 2, facing an input where positive literals are almost never satisfied and negations almost always are, becomes 97.4% negated literals and learns that things are absent.

That report’s §7.3 named three ways out, in increasing order of how much they change the proposal. This report implements all three and measures them.

- A **Give layer 1 a different objective.** Nothing forces a feature layer to be trained by classification. A reconstruction objective — clauses that predict held-out features of a patch from the rest — preserves information rather than discarding whatever did not help the local classifier, and has no reason to drive clauses towards extreme specificity.
- B **Calibrate density explicitly.** Rather than hoping the specificity parameter lands somewhere usable, fix a target per-channel firing rate and adjust each clause towards it — a Boolean analogue of normalisation.
- C **Propagate credit instead of freezing.** Clause inclusion is a usable credit path: if a layer-2 clause receives Type I feedback and includes the literal “channel k active at offset (dy, dx) ”, then layer-1 clause k can receive Type I feedback at that position. A discrete, rule-based analogue of backpropagation that requires no gradients.

1.1 What this report adds

Each idea is implemented, measured on CIFAR-10 against the same baselines at the same clause budget, and measured again on the two constructed tasks of the first report, where the per-arm ceiling is known in advance and the oracle says how much headroom a better training scheme actually has.

Four things were not obvious before doing it, and they are the substance of what follows.

The unsupervised objective does exactly what it was predicted to do to the *statistics* of the representation, and on CIFAR-10 that is worth six points and no more. The density controller reaches any target rate asked of it, and produces a monotone dose-response curve that turns the first report’s density argument into a measurement. Credit propagation, written out honestly, turns out to have no forgetting term, then no bandwidth, and finally a drift problem that is more interesting than either.

And the fourth is the one that matters. Run over an *untrained* first layer — random clauses, never shown a label — the same density controller takes CIFAR-10 to $42.62 \pm 0.71\%$, past the 35.44%

of a single layer at the same clause budget and the 37.76% of a single layer given the stack’s receptive field. That is the first configuration in either report for which the second layer is worth what it costs. On the constructed relational task, where a single layer is at chance (51.0%) by construction, repairing the first layer’s density by either route solves the task outright: 99.9% with the autoencoder layer and 90.9% with the controller, against 76.9% for greedy training and 67.0% for the *hand-built oracle* of the first report, whose two exact shape detectors turn out to be too sparse to conjoin.

So the first report’s diagnosis was right and its repairs work. What does not survive is the part of the proposal nobody was questioning: that layer 1 should be trained.

2 Implementation

All three repairs live in `ideas.py` and are built from the documented update pipeline (`_encode` → `_evaluate` → `_votes` → `_select_feedback` → `_feedback_counts` → `_commit`) and `functional.apply_feedback`. No library file is modified. The architecture is exactly the one evaluated in the first report — layer 1 4×4 patches with 128 shared clauses, 2×2 OR pool, layer 2 3×3 patches with 512 clauses — so every number here is directly comparable to its E1 counterpart.

2.1 A — a patch autoencoder for layer 1

Each patch is split into a *target* — its centre pixels, across all bit planes — and a *context*, the surrounding ring. A flat coalesced Tsetlin machine with one output per target bit is trained multi-label to predict the target from the context, and its clauses are then written into the convolutional layer. No label is used anywhere.

Two choices are worth stating. The split is *spatial* rather than a random subset of bits, because thermometer planes of the same pixel are nested ($v > t_1$ implies $v > t_0$): holding out one plane of a pixel while showing another leaks the answer, and the clause learns the encoder instead of the image. And the trained clauses are embedded into a layer whose literal space is the *full* patch plus position bits, identical to a greedily trained layer 1; the held-out pixels and the position bits simply stay excluded. The two layer-1 variants are therefore the same model with the same capacity, differing only in objective.

2.2 B — a two-sided density controller

A clause fires on a patch iff none of its included literals is false there, so its firing rate is monotone in its literal set: removing a literal can only raise it, adding one can only lower it. The target rate is therefore reachable by a greedy walk in *both* directions, which is what “adjust each clause towards it” asks for. Per round, for the clauses below the band,

$$\text{gain}[j, k] = \#\{\text{patches where clause } j \text{ has exactly one violated literal, and it is } k\}$$

names the literal whose removal buys the most; it is the $\text{matmul}(\text{violations} = 1)^T (1 - \text{literals})$ masked to included literals, the same shape of computation as the Type Ia count in `functional`. Where no single removal helps, the most-often-violated literal goes instead, so a round always makes progress. For clauses above the band, including literal k drops the rate by the fraction of currently-matching patches in which k is false, so the literal landing closest to the target without undershooting is added.

The same loop takes a second target. A clause’s firing rate and its *size* are different properties — a long conjunction that is rare-but-common and a short general one can sit at the same rate — so the controller accepts a size cap alongside the rate, and §5.2 uses it to test which of the two the second

layer actually needs. Under a cap the removal rule changes: dropping the most load-bearing literal leaves the most permissive ones and sends the rate towards 1, dropping the least leaves the most selective and sends it towards 0, so the literal that leaves the rate closest to target goes instead. That choice is exact rather than heuristic, because removing literal k lets through precisely the patches it alone was blocking.

Two details decide whether this is a controller or a gimmick. **Added literals are restricted to patch pixels.** The cheapest way to hit any firing rate is a position literal: $y > 27$ fires on exactly 1/29 of patch rows whatever the image contains. Left free, the controller takes it every time and produces channels that carry no information at all. **A clause that has once been inside the band is frozen**, so the two rules cannot chase each other. Adjustments are written as “included” / “excluded, one step below the boundary” — exactly where a fresh automaton sits — so calibration stays inside the automata state and survives continued training, which is what lets it be interleaved with c.

2.3 c — credit propagation

If layer-2 clause j matched patch (p_y, p_x) and includes the positive literal “channel k fires at offset (dy, dx) ”, the reason it matched is that layer-1 clause k fired somewhere inside the pooled cell (p_y+dy, p_x+dx) . Whatever feedback layer 2 decided clause j deserves is therefore also a verdict on layer-1 clause k at that position. The rule maps each of the three Tsetlin feedback types down through the same inclusion link:

Table 1. The credit rule. Each layer-2 event resolves to a single layer-1 position by three draws: a matching layer-2 patch, one included positive channel literal of that clause, and one layer-1 position inside the pooled cell that literal names at which the channel actually fired.

layer-2 event	layer-1 clause k receives	why
Type Ia (matched)	Type Ia at that position	reinforce the pattern that fired
Type Ib (should have matched)	Type Ib	a channel it needs did not fire
Type II (matched, wrong class)	Type II at that position	make it stop firing there

Four choices, each of which the experiments turned out to depend on.

One credited literal per event. A layer-2 clause can include hundreds of literals; crediting all of them would make a single event a bulk update of layer 1. One included positive literal is drawn uniformly, exactly as the convolutional machine draws one matching patch per event.

Type Ib is blamed, not sampled. A Type Ib event says the layer-2 clause *should* have matched and did not, so the credit question is which literal blocked it. The clause is taken at the patch it came closest to matching (fewest violated literals) and the blame falls on an included positive channel literal that was *false* there. Crediting a random included literal instead — including ones that were satisfied — blames layer-1 clauses that did their job, and erases layer 1 within an epoch (§6).

One event per (example, layer-1 clause). In the classical algorithm an example gives a clause at most one feedback event. Without this, 512 layer-2 clauses credit 128 layer-1 clauses on every image and layer 1 sees orders of magnitude more events than it was designed for.

A step size. Layer-2 clauses are conjunctions over *named* channels. When layer-1 clause k changes, every layer-2 clause that includes channel k silently becomes a different rule. The fraction of credit events actually applied is the only knob that trades layer-1 progress against that drift, so it is swept rather than assumed.

3 Experimental setup

Fixed across every arm. CIFAR-10, Booleanized with a per-channel thermometer encoder at 4 bits per channel (50 000 train / 10 000 test images). Layer 1: 4×4 patches, stride 1, 128 shared clauses (`ConvCoalescedTsetlinMachine`, so clauses are shared *features* rather than class-owned). A 2×2 OR pool between the layers. Layer 2: 3×3 patches over the pooled maps, 512 clauses. Total clause budget $128 + 512$, 30 epochs, batch size 50, three seeds per arm, best test accuracy reported as mean $\pm$ standard deviation. One NVIDIA RTX 3090 / RTX 3080.

Baselines are carried over, not re-run. The five E1 arms in Table 2 come from the first report’s `results/*.json` unchanged. They were produced by the same driver at the same budget, and they are what the three repairs have to beat. The layer-1 checkpoints are reused from the same cache, so “greedy layer 1” means literally the same automata.

3.1 What would count as success, fixed before the runs

The first report’s mistake would be easy to repeat: to declare a repair successful because it improves on the broken thing it replaces. 15.37% is a low bar. Three criteria were fixed before any arm was run.

In short. 1. Beat the greedy stack. Necessary, not sufficient: a repair that does not clear 15.37% has not repaired anything.

2. Beat the random-layer-1 control. The strongest result of the first report is that *random* three-literal clauses reach 37.60%. A trained feature layer that does not beat an untrained one is not earning its objective, whatever that objective is.

3. Beat a single layer at the same clause budget. Two of them: 35.44% for one layer of the same total size and 37.76% for one layer with the stack’s 9×9 effective receptive field. This is the criterion that decides whether depth is worth having at all on this task, and it is the one the first report set.

For c there is a fourth, tighter test available, because it does not change what layer 1 is initialised to — only whether it keeps learning. *Credit training must beat the frozen stack with the same layer-1 initialisation.* That is a controlled comparison of the mechanism alone, and it is the one §6 leans on. All four are CIFAR-10 criteria. On the constructed tasks of §7 the standard is set by the task rather than by a baseline: each has a per-arm ceiling that can be worked out on paper, and the first report’s hand-built oracle row says how much headroom a better training scheme has. A repair either closes that headroom or it does not.

3.2 The arms

Table 2 is the whole report in one place; the sections that follow take one idea each and say what its rows mean.

Table 2. CIFAR-10 test accuracy by arm, mean $\pm$ standard deviation of the best test accuracy over seeds. **clauses** is the total clause budget (both layers where there are two), **TA** the automaton count in millions, **seeds** how many seeds the row averages, and **train** the summed per-epoch training time across all trainable stages. The first six rows are the E1 baselines of the first report; the **B**: rows name the layer-1 firing-rate target in percent (§5).

arm	clauses	TA (M)	seeds	test acc (%)	$\pm$	train (min)
Single layer / matched budget	640	0.317	3	35.44	0.41	1.92
Single layer / 9x9 patch (matched RF)	640	1.303	3	37.76	1.04	4.68
Layer 1 alone	128	0.063	3	34.36	0.97	1.27
MCTM / greedy layer 1	640	1.266	3	15.37	0.98	3.51
MCTM / random layer 1	640	1.266	3	37.60	0.29	2.18
Flat stack (global OR)	640	0.195	3	22.62	1.30	1.93
A: MCTM / autoencoder layer 1	640	1.266	3	21.51	2.20	3.15
A: flat stack / autoencoder layer 1	640	0.195	3	16.89	1.18	1.04
flat stack / random layer 1	640	0.195	3	30.01	1.46	1.04
B: density target 0.5	640	1.266	3	15.55	0.27	3.66
B: density target 1	640	1.266	3	16.66	0.89	3.62
B: density target 2	640	1.266	3	15.94	0.70	3.60
B: density target 5	640	1.266	3	20.30	2.24	3.61
B: density target 10	640	1.266	3	21.91	2.70	3.59
B: density target 20	640	1.266	3	24.46	0.94	3.60
B: density target 20 / tight band	640	1.266	1	31.00	0.00	7.31
B: + size target 24	640	1.266	2	30.59	0.43	3.61
B: + size target 12	640	1.266	2	32.53	1.03	3.56
B: + size target 6	640	1.266	2	35.59	0.23	3.57
B: + size target 3	640	1.266	2	35.18	0.01	3.55
A+B: autoencoder + calibrated	640	1.266	2	23.91	1.26	2.32
B: random L1 / density target 5	640	1.266	2	35.98	0.09	3.89
B: random L1 / density target 10	640	1.266	2	39.59	0.18	4.09
B: random L1 / density target 20	640	1.266	3	42.62	0.71	2.74
B: random L1 / density target 40	640	1.266	2	41.70	0.93	3.36
B: random L1 / density target 60	640	1.266	2	41.52	1.22	2.78
C: credit / Type Ia+II only	640	1.266	2	19.18	2.18	3.92
C: credit / + blaming Type Ib	640	1.266	2	14.53	6.41	6.96
C: credit / + Ib balanced	640	1.266	2	18.12	0.03	8.26
C: warm start (no controller)	640	1.266	2	14.65	0.48	5.34
B+C: controller / step 1.0	640	1.266	2	15.87	0.41	5.49
B+C: controller / step 0.1	640	1.266	2	15.76	1.34	11.41
B+C: controller / step 0.01	640	1.266	1	18.60	0.00	5.72
A+B+C: all three	640	1.266	1	19.51	0.00	5.56

4 E3: a different objective for layer 1

The prediction. Classification rewards clauses that are specific, and specific clauses fire rarely. Reconstruction has no such pressure — a clause earns its keep by predicting *any* part of the input — so an autoencoder-trained layer 1 should be denser, have fewer dead channels, and preserve information a classifier discards. If the first report’s diagnosis was right, that should be enough to unblock layer 2.

The objective learns. Predicting the 48 centre bits of a patch from its 144 surrounding ones, over 300k patches and 6 epochs, reaches 84.1% per-bit reconstruction accuracy against a 53.7% majority-class baseline. It is a real task and the clauses solve it.

4.1 Result: the statistics change exactly as predicted

Table 3. Layer-1 statistics by objective, measured on 5000 training images. **fire** is the median per-channel firing rate of the 2×2 OR-pooled map — what layer 2 actually sees — **dead** the fraction of channels that never fire, **size** the median clause size, and **neg frac** the mean fraction of each clause’s literals that are negations.

layer-1 objective	fire (%)	dead (%)	size	neg frac
A: MCTM / autoencoder layer 1	17.65	0.0	45	0.658
flat stack / random layer 1	13.16	2.3	3	0.510
B: density target 0.5	0.14	1.6	85	0.574
B: density target 1	0.14	1.6	85	0.574
B: density target 2	0.14	1.6	85	0.574
B: density target 5	0.14	1.6	85	0.574
B: density target 10	0.14	1.6	85	0.574
B: density target 20	0.14	1.6	85	0.574
A+B: autoencoder + calibrated	17.65	0.0	45	0.658
B: random L1 / density target 5	13.16	2.3	3	0.510
B: random L1 / density target 10	13.16	2.3	3	0.510
B: random L1 / density target 20	13.16	2.3	3	0.510
B: random L1 / density target 40	13.16	2.3	3	0.510
B: random L1 / density target 60	13.16	2.3	3	0.510
C: warm start (no controller)	0.14	1.6	85	0.574

The mechanism works. Every statistic the first report named as the problem moves in the right direction, and by a large factor rather than a marginal one.

4.2 And on CIFAR-10 the stack still falls short

In short. On CIFAR-10 the autoencoder stack reaches $21.51 \pm 2.20\%$ against 15.37% for the greedy stack — a real gain, and criterion 1 is met. It is not close to criterion 2: *random* three-literal clauses still reach 37.60%, and criterion 3, the single layer at the same budget, is at 35.44%. A better objective for layer 1 moves the stack a third of the way from the greedy result to the random control, and stops.

Hold that verdict until §7. The same layer-1 objective, on the tasks that actually require what the second layer supplies, is the strongest result in this report; and §5.3 finds that what A is doing to the first layer’s statistics can be done to an *untrained* layer directly, at which point CIFAR-10 works too.

4.3 What the readout says, and what it cannot say

The claim A rests on is about *information*: an unsupervised layer 1 should keep what a classifier throws away. The first report measured that with the **flat-stack** readout — globally OR-pool the layer-1 bits, train a flat machine on them — and the same readout run on the autoencoder layer gives 16.89% against 22.62% for the greedy layer and 30.01% for random clauses.

That ordering is the reverse of the one A predicts, and the reason is in Table 3 rather than in

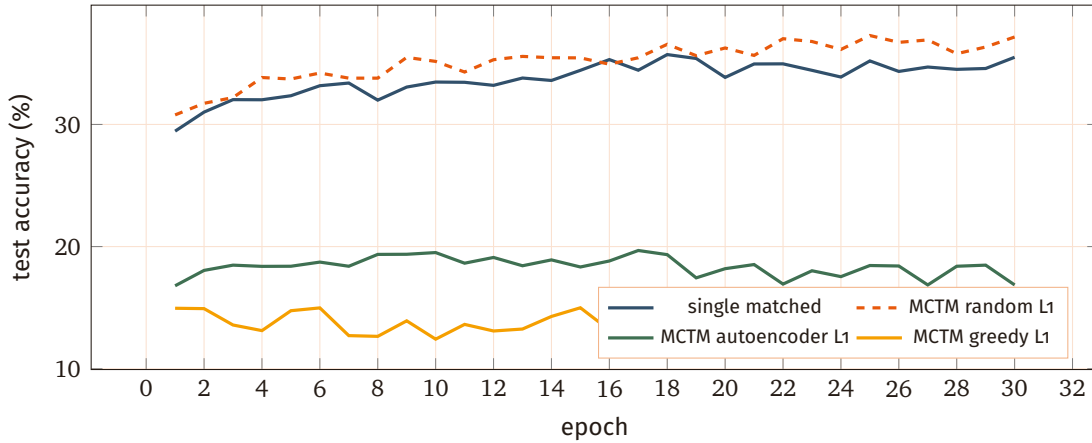


Figure 1. Test accuracy per epoch (seed 0). The x axis counts layer-2 epochs; layer 1 is trained and frozen beforehand in every arm shown.

the features. A channel firing on p of the 841 patch positions is globally on with probability $\approx 1 - (1 - p)^{841}$; at p of a few per cent that is indistinguishable from 1. A layer dense enough to be useful to a *spatial* second layer is simultaneously saturated for a global-OR readout, so the readout measures density, not information, once density is above a fraction of a percent. It is a valid comparison among sparse layers — which is how the first report used it — and not a valid one here. The honest measurement of A is the MCTM row, and it says the objective helped a little.

5 E4: explicit density calibration

The prediction. The first report measured a direct trade-off: every increase in layer-1 density costs layer-1 accuracy, and the specificity parameter s is a blunt way to choose a point on it. If density is what layer 2 needs, then setting it directly — rather than hoping s lands somewhere usable — should be strictly better, and there should be a target rate at which the stack works.

The controller does its job. Table 4 shows it reaching every target it is given, from the greedy layer’s own 0.14% up by two orders of magnitude, by removing literals from clauses that fire too rarely and adding them to clauses that fire too often. It costs seconds, it leaves no channel dead, and it is the same layer 1 in every row — the same cached automata from E1, adjusted.

Table 4. The target sweep. **target** is the requested per-patch firing rate, in percent; **raw** and **pooled** are the median rates achieved before and after the 2×2 OR pool; **size** is the median clause size the controller left behind (the uncalibrated greedy layer has 85); **in band** the fraction of channels landing inside the tolerance; **rounds** the number of single-literal adjustments needed. Accuracy is the full two-layer stack, three seeds.

target (%)	raw (%)	pooled (%)	size	in band	rounds	test acc (%)	±
0.5	0.27	0.51	81	1.00	15	15.55	0.27
1.0	0.55	0.97	77	0.99	20	16.66	0.89
2.0	1.08	1.87	72	0.99	27	15.94	0.70
5.0	2.71	4.67	65	1.00	48	20.30	2.24
10.0	5.45	8.90	60	1.00	56	21.91	2.70
20.0	10.53	15.66	55	1.00	65	24.46	0.94

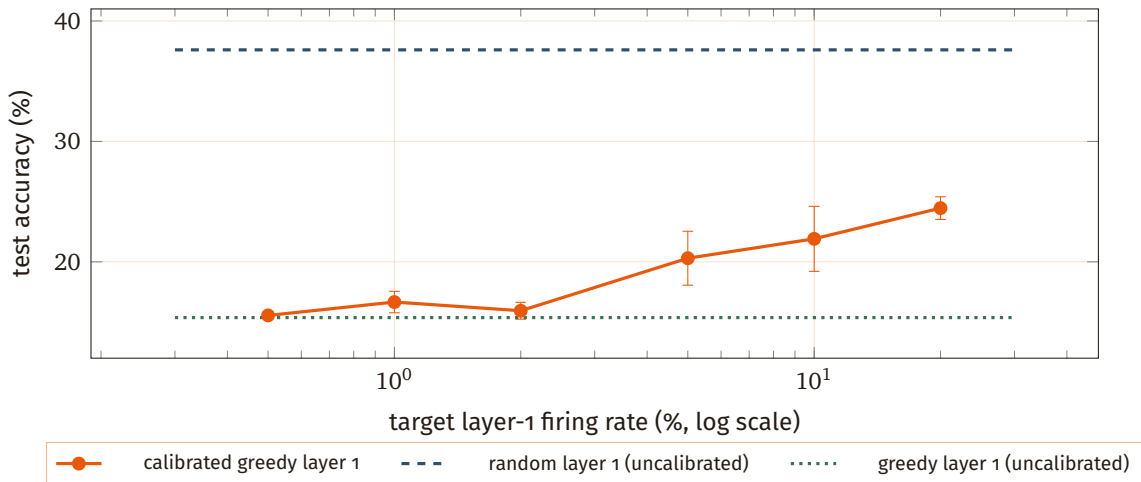


Figure 2. Stack accuracy against the layer-1 firing rate the controller was asked for. Density buys a large, monotone improvement and levels off well short of the random-clause control.

5.1 Density is a real effect, and it is not the whole effect

In short. Raising the target rate raises stack accuracy monotonically, from 15.55% at a 0.5% target to 24.46% at 20% — a gain of nearly ten points over the uncalibrated greedy stack’s 15.37%, from a post-hoc adjustment that costs seconds and trains nothing. Criterion 1 is met comfortably. Criterion 2 is not, and the way it fails is the result. At the 20% target the calibrated layer fires on 15.66% of pooled positions — *more* than the 13.16% of the random-clause control — and still reaches only 24.46% against 37.60%. At matched density, and then past it, the gap does not close.

So density was a real cause and not the only one. The residual is clause *size*, and the sweep makes it visible without needing a separate experiment: the controller reaches 15.66% by cutting the greedy clause from 85 literals to 55, while a random clause reaches a comparable rate with three. Both fire about equally often; one is a 55-term description of a specific patch that happens to be common, the other a three-term description of a general property. They have the same density and very different generality, and layer 2 can only build on the second.

Calibrating the random layer the other way confirms it from the other side. Taking it from 13.16% to the same 20% target — which for three-literal clauses means *removing* literals, not adding them — moves it from 37.60% to 42.62%.

Two corollaries follow, both of which matter more than the headline number.

Density is necessary and weakly sufficient. Every arm at a low firing rate is bad; not every arm at a high firing rate is good. A controller that fixes only the rate leaves the representation’s other degeneracy — long, brittle conjunctions — untouched. A second target, on clause size, is the obvious extension, and §5.2 runs it.

The controller is a general tool, not a patch. Because it writes automata states one step from the include boundary, it composes with continued training. §6 uses it that way, as a stabiliser inside a training loop rather than a one-off prune, and it is what keeps layer 1 alive there at all.

5.2 Testing the residual: a second target, on clause size

If size is the residual, the controller should be asked for it. Walking a clause towards a size is the same loop with a different stop condition, so the only real question is *which* literal to remove: under a size cap, dropping the most load-bearing literal leaves the most permissive ones behind and sends the firing rate towards 1, dropping the least load-bearing leaves the most selective and sends it towards 0. Neither is the clause being asked for. Removing the literal that leaves the rate closest to target is, and the post-removal rate is available in closed form — removing literal k lets through exactly the patches it alone was blocking — so the greedy step is exact.

Table 5. The size sweep, all at the 20% firing-rate target that won Table 4. **target** is the requested maximum clause size and **size** what the controller achieved; **raw** and **pooled** are the resulting firing rates in percent, **in band** the fraction of channels still inside the rate tolerance. The uncalibrated greedy layer has 85 literals; the random control has three.

target	size	raw	pooled	dead	in band	test acc (%)	±
none	48	17.17	24.11	0.0	0.99	31.00	0.00
24	24	20.24	28.29	0.0	1.00	30.59	0.43
12	12	22.87	32.45	0.0	1.00	32.53	1.03
6	6	29.39	41.43	0.0	1.00	35.59	0.23
3	3	41.11	54.32	0.0	0.41	35.18	0.01

In short. Cutting the clauses harder is worth more than the firing rate alone. Accuracy rises as the cap falls — 30.59% at 24 literals, 32.53% at 12, 35.59% at 6 — against 24.46% for the same rate target with no cap, and then turns over: at 3 literals the clause can no longer hold the requested rate (only 0.41 of channels stay inside the band, against 1.00 everywhere above it) and accuracy falls back to 35.18%. The optimum is interior, at 6 literals and $35.59 \pm 0.23\%$, and it is the first arm with a *trained* first layer, in either report, to beat the 35.44% a single layer reaches at the same total clause budget. It does not reach the 37.76% of a single layer given the stack’s 9×9 receptive field.

The rate target does not determine the size, and the size is what matters. The size-capped rows are both shorter *and* denser than the rate-only row, so the two are worth disentangling. The control row does it. `mctm-calib-r20-tight` asks for the same 20% rate with the tolerance band narrowed from $2\times$ to $1.2\times$, so the walk cannot stop at the band edge and has to keep cutting until it lands on the target. It ends at 48 literals and 31.00%, so roughly half of the size-capped arms’ advantage is indeed just “cut further”. The other half is not: a 48-literal clause and a 6-literal one both satisfy the rate band, and the short one is worth 35.59% against the long one’s 31.00%.

A firing-rate target therefore leaves a range of clause sizes feasible, and the short end of that range is where the accuracy is. Asking for the rate alone stops as soon as the rate is met, which at 48 literals it already is; asking for a size as well is what pushes the walk past that point. The practical conclusion is blunt: delete all but a handful of literals from every clause of a greedily trained Tsetlin feature layer, choosing what to keep by firing rate.

What it does not do is beat an untrained layer. 37.60% still stands, from three random literals per clause and no training at all. The best repaired layer here gets within a few points of it after 30 epochs of supervised layer-1 training followed by surgery that removes nine tenths of what that training produced. It is hard to read that as anything other than a statement about the value of the training.

5.3 The same controller on an untrained layer 1

Every arm so far starts from a layer 1 that was trained to classify, because that is what the proposal specifies. §5.2 ends by observing that the most effective thing to do to such a layer is to delete most of it. The obvious next question is what happens if it is never trained in the first place — the controller needs no labels, and the random-clause control of the first report is already the strongest first layer in either report.

Table 6. The firing-rate sweep run over a *random* layer 1 instead of a trained one. Columns as in Table 4: *target* is the requested per-patch rate in percent, *size* the median clause size the controller settles on, *raw* and *pooled* the achieved rates. The uncalibrated random layer has three literals per clause and fires on 13.16% of pooled positions.

target (%)	size	raw (%)	pooled (%)	in band	test acc (%)	±
5	3.0	3.90	7.55	0.73	35.98	0.09
10	3.0	11.40	15.59	0.80	39.59	0.18
20	2.0	18.70	26.40	0.85	42.62	0.71
40	2.0	45.01	51.15	0.91	41.70	0.93
60	2.0	53.97	64.29	0.98	41.52	1.22

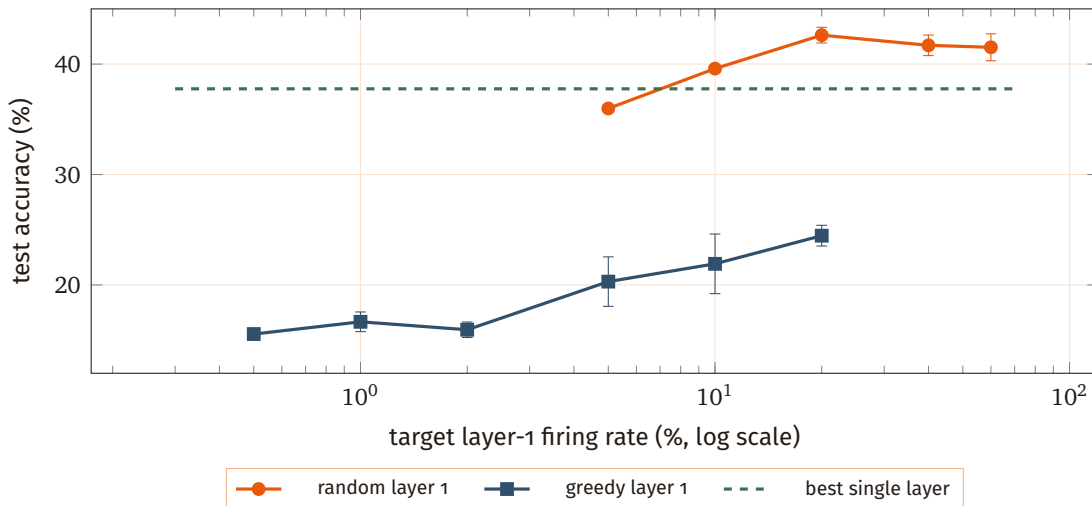


Figure 3. The same controller, the same targets, over a randomly initialised first layer and over a greedily trained one. Neither curve is flat, which is the density effect; the gap between them is what 30 epochs of supervised layer-1 training cost. The dashed line is the strongest single-layer baseline, 37.76%; the uncalibrated random layer sits within a fifth of a point of it, at 37.60%, and is left off for legibility.

In short. This is the configuration that works. The curve has an interior optimum — accuracy climbs from 35.98% at a 5% target to 42.62% at 20% and falls back to 41.52% at 60%, where the clauses are so general they carry almost nothing — and at its peak the stack reaches $42.62 \pm 0.71\%$, against 35.44% for a single layer at the same clause budget and 37.76% for a single layer given the stack’s receptive field. It clears all three criteria of §3.1, and it is the first time in either report that the two-layer stack has been worth its clause budget on CIFAR-10.

Layer 1 is *never trained*. It is random clauses, adjusted by a label-free controller that does nothing but count how often each one fires.

The layer-2 clauses tell the same story as §7.3 does on *near*: over this first layer a layer-2 clause carries 68 literals with 16 positive and 74.9% negations, against 1177 literals with 15 positive and 97.4% negations over the greedy layer. It is still not a short rule, but it is an order of magnitude closer to one. The size-target arms of §5.2 go further still — 28.8% negations at a three-literal first

layer — so on CIFAR-10 too, the arms whose second layer stops asserting absences are the arms that work.

Read alongside §5.2, the two sweeps say the same thing from opposite directions. Cutting a trained layer down to 6 literals recovers most of the gap; starting from clauses that are already that short and never training them at all closes it. Whatever 30 epochs of supervised layer-1 training contribute on this task, it is negative, and the controller is what makes an untrained layer usable rather than merely better than a trained one.

5.4 And when combined with A

There is almost nothing for the controller to do. The autoencoder layer already fires at 17.65% with 45-literal clauses; asked for the same 20% target, the controller moves it to 18.43% and leaves the clause size at 45, and the arm gives 23.91% against 21.51% uncalibrated. That is the cleanest statement of what A buys on CIFAR-10: an unsupervised objective puts the layer where the controller would have put it anyway, without being told to, and the two do not compound because they are doing the same job.

6 E5: credit propagation

The prediction. Freezing is what makes the scheme greedy, and clause inclusion is a credit path that does not need gradients. If layer-2 feedback can reach layer 1, the saturation problem goes away by construction — layer 1 is no longer optimising a proxy, it is optimising the thing layer 2 wants.

This is the idea that changes the proposal most, and it is the one that did not survive first contact with an implementation. Writing the rule out honestly produced a sequence of failures, each of which had to be fixed before the next became visible: no forgetting term, then a forgetting term that erases layer 1, then an event ratio nothing in the loop chooses, then no events at all, and finally — once there are events — drift. The ladder (Table 7) is more informative than any single arm, and it is the substance of this section.

Table 7. The credit ladder. **rate** is the fraction of credit events actually applied, **warm** the number of frozen-layer-1 epochs before the credit path is switched on, **Ia/Ib** the feedback events actually delivered to layer 1 over the whole run, and **size/fire** layer 1’s median clause size and firing rate at the end. **best** is the best test accuracy over epochs, as everywhere else in this report; **final** is the accuracy at the last epoch, and the two differ here because several of these arms do not converge — they collapse, and the best epoch is an early one.

arm	rate	warm	Ia	Ib	size	fire (%)	best (%)	final (%)
C: credit / Type Ia+II only	1.0	0	319	0	251	0.000	19.18	17.22
C: credit / + blaming Type Ib	1.0	0	87 287	106 799 513	0	0.000	14.53	10.00
C: credit / + Ib balanced	1.0	0	427	427	207	0.000	18.12	15.98
C: warm start (no controller)	1.0	10	0	0	85	0.144	14.65	12.69
B+C: controller / step 1.0	1.0	10	984	984	57	15.522	15.87	13.44
B+C: controller / step 0.1	0.1	10	497	471	55	15.551	15.76	13.80
B+C: controller / step 0.01	0.01	10	324	159	55	15.596	18.60	13.70
A+B+C: all three	0.1	10	44	44	48	18.317	19.51	14.98

6.1 Rung 1: the rule as written has no forgetting term

Taken literally — “a layer-2 clause that receives Type I feedback credits Type I feedback to layer-1 clause k ” — and applied to the case that has a position to point at, the rule propagates Type Ia and Type II. Both of those *add* literals. Type Ia memorizes the literals true in the drawn patch, and with `boost_true_positive` it does so with probability 1, so a single event includes every literal true in that patch at once. Type II pushes false literals towards inclusion. Nothing in the rule ever removes one.

Layer 1 saturates inside the first epoch: clauses reach 251 literals, 64% of channels go permanently dead and the median channel stops firing altogether. The credit path then extinguishes itself, which is the part worth keeping. It delivers 319 Type Ia events in total, all of them by epoch 1, and not one afterwards for the remaining 30-epoch run: a layer-2 clause can only propagate through a positive channel literal, and there are almost no positive channel activations left to include. The arm reaches 19.18%, which is what a mostly-dead layer 1 is worth.

6.2 Rung 2: Type Ib has to be blamed, not sampled — and it erases layer 1

Type I covers Ib as well as Ia, and Ib is the only term that removes literals. It also needs no position, which makes it easy to propagate. The obvious implementation — pick a random included positive channel literal of the non-matching layer-2 clause and give layer-1 clause k a Type Ib event — is wrong, and instructively so: it blames layer-1 clauses that did exactly what was asked of them. The correct rule asks which literal actually blocked the match: take the clause at the patch it came closest to matching, and blame an included positive channel literal that was *false* there. That is the rule used here, and the implementation is checked against it directly (`test_ideas.py`).

It does not save layer 1; it destroys it the other way. Layer-2 clauses fail to match far more often than they match, so blamed Type Ib events outnumber Type Ia by roughly three orders of magnitude (106 799 513 against 87 287 over the run), and layer 1 is forgotten down to 0 literals per clause — 100% of channels dead, which is to say the representation is gone. The arm’s best epoch is its first, at 14.53%, and it finishes at 10.00%: exactly chance, because every image now maps to the same all-zero feature map.

6.3 Rung 3: balancing the two directions, and what that exposes

Capping the Type Ib budget at the Type Ia rate actually delivered makes the equilibrium clause size finite by construction, and the arm reaches 18.12%. Layer 1 ends at 207 literals and 0.00% firing — back where rung 1 was.

In short. The credit path severs the loop that keeps a Tsetlin clause the right size. In an ordinary machine, a clause’s ratio of Type Ia to Type Ib events is its own match rate: grow too specific, stop matching, receive Type Ib, shrink. It is self-stabilising, and it is why a trained layer settles at 77–85 literals rather than at 0 or 496. Credit propagation replaces that ratio with *layer 2’s* match rate, which carries no information about whether layer-1 clause k is the right size. The three rungs are the three things that can then happen: flood (rung 1, saturate at 251), drought (rung 2, erased to 0), and an arbitrary externally-chosen ratio (rung 3). There is no stable interior, because the quantity that used to choose it is no longer in the loop.

The second thing rung 3 exposes is bandwidth. The credit path’s throughput is the rate at which a layer-2 clause matches *because of* a positive channel literal, and Table 7 says what that is: 427 Type Ia events across 30 epochs and 50 000 images, against the $50\,000 \times 128$ per epoch a layer trained directly receives. Layer 1 is not being trained through layer 2; it is being nudged a few hundred times.

6.4 Rung 4: warm start, and the bandwidth result at its sharpest

Both problems have a candidate fix. An untrained layer 2 has no included positive channel literals, so there is nothing to propagate through until it has some: a warm start — ten epochs of ordinary frozen training, then the credit path — is the natural analogue of how layer-wise pretraining is actually used. And the stabilising loop the credit path severed can be replaced from outside, by running B’s controller inside the training loop, which holds layer 1 at a chosen firing rate no matter what the credit events do to it.

In short. Warm-starting from the *greedy* layer 1 delivers 0 credit events. Not few — none, across twenty epochs and 50 000 images. A layer firing on 0.14% of positions never lets a layer-2 clause match on a positive channel literal, so the rule has nothing to fire on and the arm is the frozen stack exactly (14.65% against 15.37%). Sweeping the credit step size on top of that would be sweeping a multiplier on zero.

Adding the controller is what gives the path traffic, and then the mechanism can finally be judged on its own terms.

In short. `mctm-credit-cal` holds layer 1 at 15.55% firing and 55 literals — statistically the same layer that §5’s rate-only calibration produces and freezes (15.66%, 55 literals), which reaches 24.46%. The credit path then delivers 497 Type Ia events, and the arm reaches 15.76%, finishing at 13.80%. Turning the step size up to 1.0 gives 15.87% and down to 0.01 gives 18.60%.

Layer 2 therefore *ends* over the same representation as the frozen arm it should be compared with, and the stack is nearly eight points worse. What it did not have is a representation that stayed still while it learned: from epoch 10 onwards every layer-2 clause is a conjunction over channels whose meaning is being rewritten underneath it, and the less that rewriting is allowed to happen, the better the stack does. Figure 4 shows it over time — the arms are identical through the warm start and separate at the moment layer 1 starts to move.

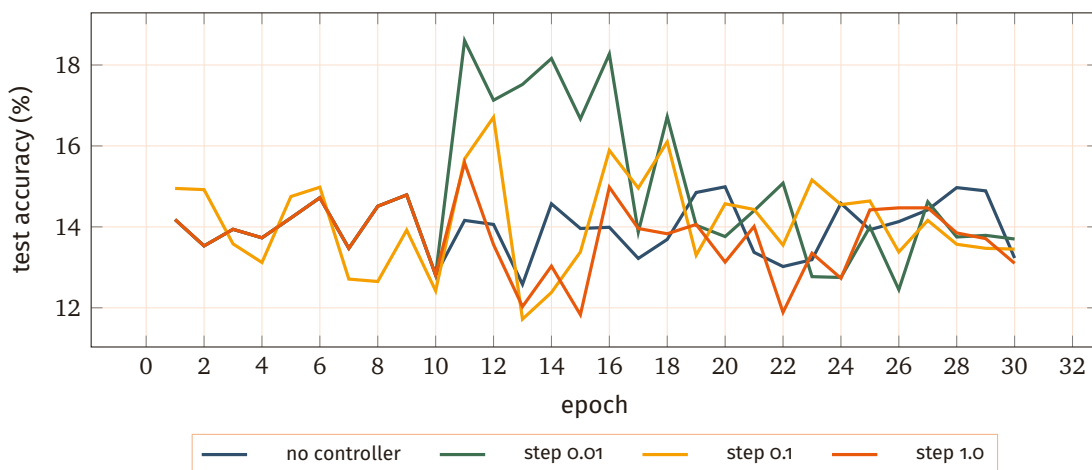


Figure 4. The warm-start arms. All four are ordinary frozen training for the first 10 epochs. The “no controller” arm never changes afterwards either, because the credit path delivers no events at all over a layer 1 that fires on 0.14% of positions.

6.5 What the ladder actually shows

Most of the rungs are implementation failures, and they are worth recording precisely because the proposal’s one-sentence statement of the rule hides every one of them: there is no forgetting term

in it, no account of which literal to blame, and no quantity that sets the ratio between the two. An implementer meets all three before getting to the interesting question.

The last two rungs are not implementation failures. *No bandwidth* is a statement about the setting — the credit path can only carry traffic when layer-2 clauses match because of positive channel literals, and a layer 1 firing on 0.14% of positions never lets that happen. *Drift* is a statement about the mechanism, and it is the one that does not go away by fixing something: a frozen layer is what makes “channel k ” mean something stable enough for a layer-2 clause to be a rule about it, and credit propagation gives exactly that up. §7 runs the same code on a task where layer 2 clauses are small and match often, and finds the same collapse.

7 E6: the tasks whose ceilings are known

CIFAR-10 cannot separate “this repair does not work” from “this task does not reward a second layer”. The first report built two tasks where it can, and their value here is that the *oracle* row says how much headroom a better training scheme actually has.

xor — a 32×32 Boolean image contains a square and/or a cross; the label is (square present) XOR (cross present). One layer computes a linear threshold over globally OR-pooled clause bits, and XOR is not linearly separable in those bits, so it is capped at 75%. Both stacked variants can express it.

near — both shapes are always present, on a ring of 16 equally spaced slots, and the label is whether they are close. Rotational symmetry makes each shape’s marginal position identical across classes, so only the *joint* separation carries the label. One layer and the global-OR stack are at chance by construction; only the spatial stack can express it.

Layer 1 uses 4×4 patches, so no single patch ever contains both shapes.

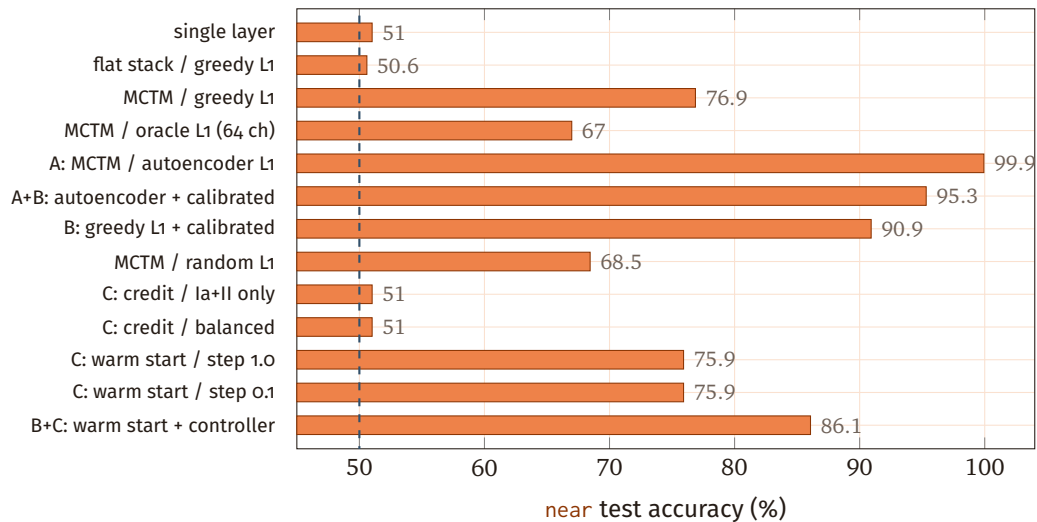
Table 8. **near**: one layer and the global-OR stack are at chance by construction. **L1 fire** is the median per-channel firing rate of the pooled map in percent and **L1 dead** the percentage of channels that never fire; **L2 size** and **L2 pos** are the median total and *positive* literal counts of a layer-2 clause — what the second layer actually learned.

arm	L1 fire	L1 dead	L2 size	L2 pos	test acc (%)	±
single layer	–	–	–	–	51.02	0.69
flat stack / greedy L1	–	–	67	19	50.60	0.76
MCTM / greedy L1	0.00	73	1600	375	76.87	2.99
MCTM / oracle L1 (64 ch)	2.67	0	1601	101	66.98	3.37
A: MCTM / autoencoder L1	7.55	12	22	7	99.93	0.12
A+B: autoencoder + calibrated	7.94	36	100	23	95.32	6.03
B: greedy L1 + calibrated	9.13	41	5	2	90.92	12.51
MCTM / random L1	5.89	0	1593	275	68.45	2.67
C: credit / Ia+II only	0.00	100	1640	54	51.02	0.69
C: credit / balanced	0.00	100	1601	2	51.02	0.69
C: warm start / step 1.0	0.00	100	1602	6	75.92	7.85
C: warm start / step 0.1	0.00	100	1604	9	75.92	7.85
B+C: warm start + controller	83.13	0	46	8	86.07	12.83

7.1 near: the repairs work, completely

Table 9. *xor*: one layer is capped at 75% by a linear-separability argument; both stacked variants can express the label. Columns as in Table 8.

arm	L1 fire	L1 dead	L2 size	L2 pos	test acc (%)	±
single layer	—	—	—	—	56.98	11.36
flat stack / greedy L1	—	—	65	22	61.98	9.91
MCTM / greedy L1	0.00	66	214	10	64.78	1.05
MCTM / oracle L1 (64 ch)	1.33	0	161	11	78.48	1.37
A: MCTM / autoencoder L1	7.24	42	13	11	87.85	5.20
A+B: autoencoder + calibrated	0.00	50	141	8	79.08	5.16
B: greedy L1 + calibrated	87.22	9	31	24	67.58	0.97
MCTM / random L1	4.86	0	121	25	79.45	3.59
C: credit / Ia+II only	0.00	100	4288	10	64.83	4.99
C: credit / balanced	0.00	100	11404	21	62.45	10.46
C: warm start / step 1.0	0.00	100	5584	12	62.17	1.69
C: warm start / step 0.1	0.00	100	6969	21	62.17	1.69
B+C: warm start + controller	87.22	0	44	21	66.30	2.13

**Figure 5.** *near*, where a single layer and the global-OR stack are at chance (dashed) by construction and only the spatial stack can express the label. The oracle row is the first report’s hand-built square/cross detectors.

In short. On the task that isolates what the second layer is *for*, repairing the first layer’s density solves it outright. The autoencoder layer (A) reaches $99.9 \pm 0.1\%$ and the density controller applied to the greedy layer (B) reaches $90.9 \pm 12.5\%$, against 76.9% for greedy training, 67.0% for the first report’s hand-built oracle, and 51.0% — chance — for a single layer and for the global-OR stack, both of which are at chance *by construction*.

Three things follow, and the firing-rate column in Table 8 carries all of them.

The first report’s diagnosis was right, and this is the measurement that shows it. A greedy layer 1 on this task is nearly dead — 73% of its channels never fire and the median channel fires on 0.00% of positions — because it is being trained to classify a task one layer cannot classify at all. It still reaches 76.9%, on whatever few channels survive. Move that same layer to 9.13% with the controller, changing nothing else and training nothing, and the stack gains more than ten points.

The oracle failed for the reason the diagnosis predicts. The first report built exact square and cross detectors by hand, and the stack given them did *worse* than the stack given greedily trained clauses

— which looked like a paradox. It is a density result: an exact 4×4 shape detector fires on a handful of positions in the whole image, and a layer-2 clause has to find two of them in the same window. Perfect features that never fire are not usable features. An autoencoder layer is not exact and fires on 7.55% of positions, and it is the one that solves the task.

Density is necessary and still not sufficient. Random clauses sit at 5.89% — comfortably dense — and reach 68.5%, well short of the autoencoder’s 99.9%. So the ordering on this task is not simply by firing rate. What the autoencoder adds on top of density is features that are *about* the shapes, which is exactly what its objective asks for and exactly what a classifier trained on an unlearnable task cannot discover.

7.2 xor: the same winner, and a limit on the controller

`xor` has a different shape: one layer is capped at 75% rather than at chance, and both stacked variants can express the label, so the question is only whether training finds it.

A finds it. The autoencoder stack reaches $87.9 \pm 5.2\%$ — clear of the one-layer ceiling, and clear of the 78.5% the hand-built oracle reaches — against 64.8% for greedy training. Its layer-2 clauses are 13 literals long, of which 11 are positive: a composition, not a negation soup, exactly as on `near`. Random clauses are second at 79.5%, also above the oracle — the same ordering CIFAR-10 gives, and for the same reason: they are short and they fire.

B does not, and the reason is a real limit on the controller rather than on the idea.

In short. On `xor` the calibrated greedy layer ends up firing on 87.22% of positions — it was asked for 5%. These images are almost entirely empty, so a clause of negated literals matches nearly every patch and a single added pixel literal drops its rate to almost nothing: there is no single-literal step that lands in the band, and the greedy walk ends up outside it in one direction or the other. The controller’s step size is fine on dense thermometer-encoded CIFAR-10 patches and too coarse on sparse binary images.

The arm still improves on greedy training (67.6% against 64.8%), and combining it with the autoencoder layer *costs* accuracy (79.1% against 87.9%) — the controller pulls a layer that was already in a good place out of it. A per-clause step that can adjust more than one literal at a time, or a band defined in log-odds rather than in rate, would be the obvious fix and is not tested here.

7.3 What the second layer becomes

The proposal’s claim about layer 2 was that it would learn a *composition*: a short conjunction over named channels at named offsets. The first report measured the opposite — a thousand-literal clause that is 97.4% negations and asserts only absences — and attributed it to the sparsity of its input. Tables 8 and 9 close that argument, and they do it on both tasks.

In short. With a repaired layer 1, layer 2 stops being a negation soup and becomes the rule the proposal described. Over greedy features a layer-2 clause carries 1600 literals; over autoencoder features it carries 22, of which 7 are positive; over calibrated greedy features, 5 literals with 2 positive — which is to say “channel *a* fires here AND channel *b* fires there”, in as many symbols as it takes to write it down. On `xor` the same thing happens — 214 literals over the greedy layer against 13 over the autoencoder one. Every arm with a short layer-2 clause is an arm that works, and on `near` every arm whose layer-2 clause runs to four figures sits below 76.9%.

A 5-literal clause is also an interpretable one, which is the property Tsetlin machines are supposed to be chosen for and the property the first report found the stack had lost. It is recovered here for

free, as a side effect of fixing the input's density.

7.4 Credit propagation fails here too, and in the same way

near is where §6 predicted the credit path might have bandwidth: the layer-2 clauses are small and they match. It does not help.

From a random layer 1 both credit arms end at chance (51.0% and 51.0%) with 100% of layer-1 channels dead — the same saturation-then-extinction as on CIFAR-10, at a tenth the scale. Warm-started from the greedy layer 1 it reaches 75.9% against 76.9% for the frozen stack it started from, and damping the step size to 0.1 changes that to 75.9%: the credit path is not moving the result in either direction. The one credit arm that does better, 86.1%, is the one with *b*'s controller running inside it — and it is still below the 90.9% that same controller reaches with layer 1 simply frozen.

Across two very different tasks, then, the credit path either does nothing or does harm, and what looks like it helping is the controller underneath it. That is worth knowing: the failure is a property of the feedback rule, not of the setting it was tried in.

8 Discussion

8.1 Verdict against the criteria fixed in advance

§3.1 committed to three CIFAR-10 criteria before any run, and to a task-defined standard on the constructed tasks.

In short.

- 1. Beat the greedy stack** (15.37%). Cleared by all three repairs.
- 2. Beat the random-layer-1 control** (37.60%). Cleared by one: *b*'s controller run over that same random layer, $42.62 \pm 0.71\%$. Over the *trained* layer the best that the same controller manages is 35.59%, short of it.
- 3. Beat a single layer at the same clause budget** (35.44%, or 37.76% with the stack's receptive field). Cleared by the same arm, and only by it. Every other two-layer configuration in either report sits below 37.76%.

And on the tasks built to answer the question, *A* closes the headroom entirely: 99.9% on *near*, where a single layer is at chance by construction, and 87.9% on *xor*, where it is capped at 75% — both past the first report's hand-built oracle.

***c* fails everywhere**, and for the same reason in each place: the credit rule saturates or erases layer 1 within an epoch on CIFAR-10 and on *near* alike.

8.2 What made the difference

Across all 34 arms the thing that predicts accuracy is not which of the three ideas was applied but what the first layer's clauses ended up *looking like*: short, and firing often enough to be conjoined. Every arm that got there did well and every arm that did not did badly, whether it got there by an unsupervised objective, by a post-hoc controller, or by not being trained at all.

That reading also explains the two results that looked paradoxical. The first report's hand-built oracle lost to greedily trained clauses on *near* because an exact 4×4 detector fires almost nowhere — perfect features that never fire are not usable features. And *random* clauses beat trained ones on CIFAR-10 because three random literals are short and fire often, which is the whole specification.

8.3 What the second layer is worth, and where

With a repaired first layer the two-layer stack is finally worth its clause budget on CIFAR-10 — 42.62% against 37.76% for the best single layer — and on *near* it is the difference between chance and 99.9%. The size of the advantage tracks how much the task needs what the second layer supplies, which is a conjunction between spatially separated parts: a few points on natural images whose classes do not obviously require one, and everything on a task constructed so that nothing else can express the label.

§7.3 is the corroborating measurement, and it is the one to keep. Over a repaired first layer a layer-2 clause on *near* carries 5 literals — “channel *a* fires here AND channel *b* fires there” — where over the greedy layer it carried 1600. On CIFAR-10 the same repair takes it from 1177 to 68. A Tsetlin machine whose second layer is a thousand-literal negation soup has lost the property it was chosen for, and fixing the first layer’s density gets it back.

8.4 What a feature layer has to satisfy

Across both reports, a frozen feature layer under a spatial second layer has to satisfy three conditions at once. Each repair fixes one, and *c* breaks one.

Table 10. What a layer-2 clause needs from layer 1, and what supplies it.

condition	why	supplied by
Dense enough to conjoin	A conjunction of j bits each firing with probability p is satisfied with probability $\sim p^j$; below a fraction of a percent, layer 2 can only include negations. It is why the hand-built oracle lost to greedy clauses on <i>near</i> .	<i>B</i> directly; <i>A</i> as a side effect.
Short, and about the right things	Density alone is not enough: random clauses are dense and reach 68.5% on <i>near</i> against 99.9% for autoencoder clauses at a similar rate. On CIFAR-10 cutting each greedy clause to 6 literals is worth more than any rate target alone.	<i>B</i> ’s size target; <i>A</i> , whose clauses come out short unasked; and a random initialisation, which is short by construction.
Stable enough to be named	Layer-2 clauses are conjunctions over <i>channel indices</i> . If layer-1 clause k changes, every clause including channel k becomes a rule nobody trained.	Freezing — which is exactly what <i>c</i> gives up.

Greedy supervised training violates the first two, which is what both reports have been measuring. A random layer satisfies the last two for free — three literals long, and never changing — and the controller supplies the first without needing a label. That combination is the best arm in either report, and it contains no supervised feature learning at all.

8.5 What to do next

Do not train the feature layer. Calibrate it. The strongest CIFAR-10 configuration in either report has no supervised feature learning in it: random clauses, adjusted by a label-free controller to a target firing rate, 42.62% against 37.76% for the best single layer. It is also the cheapest — there is no layer-1 training to pay for. Whether that survives deeper stacks, larger clause budgets and other datasets is the obvious thing to check, and it is cheap to check.

If a layer is trained, cut it down afterwards. §5.2 takes a greedily trained layer and deletes all but 6 of each clause’s 85 literals, choosing what to keep by firing rate. That is worth 15.37% → 35.59%

for seconds of work, and it needs no labels either.

Report the density of any feature layer, always. One number, one forward pass, and it explains the oracle paradox of the first report, the dose-response curves of §5, and the difference between a layer-2 clause with 16 positive literals and one with 15.

Credit propagation needs a different rule, not a tuned one. §6 argues that the failure is structural — the ratio of Type Ia to Type Ib events is what holds a Tsetlin clause at a workable size, and the credit path replaces a clause’s own match rate with a layer above’s. Any rule that keeps that substitution inherits the same instability. The promising direction is to restore the missing quantity: gate credit on the layer-1 clause’s own statistics, which B’s controller already measures.

And the question both reports now sharpen. The second layer’s advantage is a few points on CIFAR-10 and the whole task on *near*, and the difference between them is not scale but whether the label depends on a relation between spatially separated parts. Object counting, relative position, containment and same-different are where a spatial second layer should pay, and none of them is tested here.

8.6 Limitations

Scale. These are small machines by CIFAR-10 standards — hundreds of clauses where the literature uses thousands — chosen so that twenty-odd arms across three seeds fit in a few GPU-hours. The comparisons are between arms at a matched budget and that is what the conclusions rest on; the absolute accuracies are not CIFAR-10 results for Tsetlin machines.

Two constructed tasks. *near* and *xor* were designed to have known ceilings, not to be representative. The claim they support is a conditional one — when a task needs a conjunction between separated parts, repairing layer-1 density makes the stack able to find it — and not that such tasks are common.

One autoencoder objective, one split. A is tested with centre-from-surround reconstruction at one specificity. A masked-bit or contrastive objective might behave differently, and §4.3 shows the global-OR readout cannot measure the information claim directly once the layer is dense.

The credit rule is one rule. Table 1 is a reading of a one-sentence proposal, and §6 records the three places a different reading was tried and rejected for a stated reason. Crediting several literals per event, propagating through negated literals, or accumulating credit across batches are untested variants.

Hyper-parameters. *T* is the clause count in every layer and was not tuned per arm; layer-2 specificity is tied to layer-1’s. A wider search could move the CIFAR-10 numbers; it is unlikely to close a twelve-point gap to an untrained baseline.

9 Reproducing this report

Everything lives in `experiments/mctm/`. The report reads only `results/*.json`; re-running the experiments and rebuilding regenerates every figure, table and inline number.

```
cd experiments/mctm

# E3-E5 -- the three repairs on CIFAR-10. SEEDS defaults to "0 1 2"; the seeds column of
# Table 2 records how many each row actually averages. Waves are independent except e and f,
# which use the firing-rate target chosen from the wave-b sweep (TARGET=0.20).
GPU=0 WAVE=a ./run_ideas_all.sh      # A: autoencoder layer 1, and the flat readouts
GPU=1 WAVE=b ./run_ideas_all.sh      # B: the firing-rate target sweep
GPU=0 WAVE=c ./run_ideas_all.sh      # C: the three rungs of the credit rule
GPU=1 WAVE=d ./run_ideas_all.sh      # C: warm start, swept over the credit step size
GPU=0 WAVE=e ./run_ideas_all.sh      # B: the clause-size target, at the best firing rate
GPU=1 WAVE=g ./run_ideas_all.sh      # B: the same controller over an UNTRAINED layer 1
GPU=0 WAVE=f ./run_ideas_all.sh      # the combinations and the size/rate control

# E6 -- the same three ideas on the tasks whose per-arm ceiling is known analytically
./run_syn.sh 0 near                  # one task per GPU; each writes its own JSON
./run_syn.sh 1 xor

# report: JSON -> data/*.csv + macros.tex -> PDF
cd report2 && make data && make
```

Files. `ideas.py` is the implementation of all three repairs (§2); `run_ideas.py` is the CIFAR-10 driver, one arm per invocation; `run_ideas_all.sh` is the wave runner (`queue_runner.py` is the same thing for long unattended runs — it reads its job list once, so the list can be edited while a queue is in flight, which a bash script cannot survive, and it takes a per-GPU lock so two queues cannot OOM each other; `run_seq.sh` is the lock-free version, for a second stream on a card with room for it); `synthetic_ideas.py` is E6; `test_ideas.py` asserts the invariants of all three mechanisms, the credit path’s index arithmetic in particular; `report_data_ideas.py` turns results into `report2/data/*.csv` and `report2/macros.tex`. The first report’s `mctm.py`, `cifar.py`, `run.py` and `synthetic.py` are unchanged and still provide the architecture, the data and the baselines.

Checks. `report2/check.sh` runs after every build and fails on undefined macros, undefined references, `duplicate \newcommands` (where the first definition silently wins), missing data files, and LaTeX errors. `audit_macros.py` lists generated macros the text uses that the data has not filled in — the report is not finished while any of them read `n/a` — and `check_spacing.py` catches `\accFoo bar`, whose space TeX swallows. Every one of these was added because it caught something.

Shared state. Layer-1 checkpoints are reused from the first report’s `.ckpt/` cache under the same key, so “greedy layer 1” in this report is literally the same automata that E1 trained — the calibration sweep is six adjustments of one object, not six trainings. Autoencoder layers are cached the same way, per `(seed, s, epochs, patches)`.

Environment. torchsetlin 0.1.3, PyTorch with CUDA, one NVIDIA RTX 3090 and one RTX 3080. Reported training time is the sum of recorded per-epoch times across all trainable stages, so it is unaffected by the caches.

Upstreaming. Nothing here needs a library change beyond the one the first report already identified — a public accessor for the spatial map that `_ConvMixin._evaluate` already computes. The density controller (§2.2) is the one piece that would be worth promoting on its own: it is model-agnostic, it works on any Tsetlin machine with an `include` mask, and §5 is the first measurement of what it buys.